RAGBAZ DESIGN · PRODUCT & ARCHITECTURE NOTE

From RAGBAZ Frog to WeftMark

Coordination, evidence and review for multi-agent software work — a repository-grounded snapshot of Frog today, a product thesis for WeftMark, and an implementation path toward a vendor-neutral open-source control plane.

Snapshot: 11 August 2026 · **Primary reading target:** tablet · **Print target:** A5 · **Repository:** `tabenius/frog` · **Current package:** `ragbaz-frog 0.2.0`

Contents

ORIENTATION

The short version

RAGBAZ Frog started as a coordination toolkit for a very concrete problem: several human and AI workers touching the same collection of repositories at the same time. It has already grown beyond a task list. It knows about agents, locks, dependencies, repositories, affected builds, event history, remote boxes, MCP, task claiming, causal logs and a live board. The next product should not become a bigger task manager. It should become the **evidence-bearing control plane around software agents**.

Product thesis. WeftMark should answer five questions that ordinary coding agents and issue trackers answer poorly when work becomes parallel: *Who is allowed to change this? What exactly is their scope? What changed? What evidence says it is correct? Who decided it is ready?*

Frog

COORDINATION FOUNDATION

Local-first task, lock, scheduling, federation and audit primitives.

WeftMark

PRODUCT DESTINATION

Change sets, evidence, semantic scope, review decisions and handoffs.

Open source

STRATEGIC CONSTRAINT

The control plane should remain usable across proprietary and open agent/model stacks.

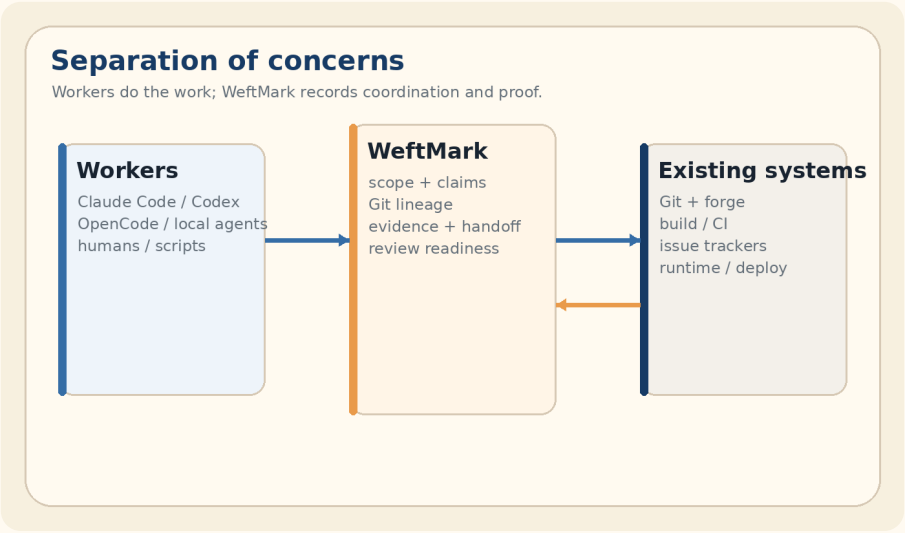

Figure 1 — Separation of concerns. WeftMark should not compete with the model, the agent loop, or the editor. Its durable value is the coordination and evidence layer that remains useful while all three layers above it change quickly.

DIDACTIC FOUNDATION

1. The problem Frog discovered

With one developer and one coding agent, coordination can be informal. A branch, a chat session and a quick diff may be enough. The situation changes when a solo developer starts several agents in parallel, or when a small team mixes different agent products. The bottleneck moves away from typing code and toward **planning, ownership, isolation, review and proof**.

The important distinction is that parallel software work creates more than one kind of conflict:

Textual conflict

Two workers edit the same lines or files. Git can detect many of these conflicts. Worktrees and branches reduce them further.

Semantic conflict

Two workers edit different files but change the same protocol, schema, security boundary, migration or behavioral contract. Git usually cannot see this.

Evidence conflict

A task says “done”, but the available proof may mean only “edited locally”, not “tested”, “CI-verified”, “reviewed” or “deployed”.

Context conflict

A handoff loses decisions, rejected options, test results, branch state or the exact next action. The next agent repeats work or unknowingly reverses decisions.

Frog was built in response to these problems rather than from a generic project-management template. That matters. Its strongest ideas are operational: task claiming is atomic, locks have owners, remote databases are not silently shared over unsafe filesystems, the scheduler understands dependencies and conflicts, and the event log can answer causal questions.

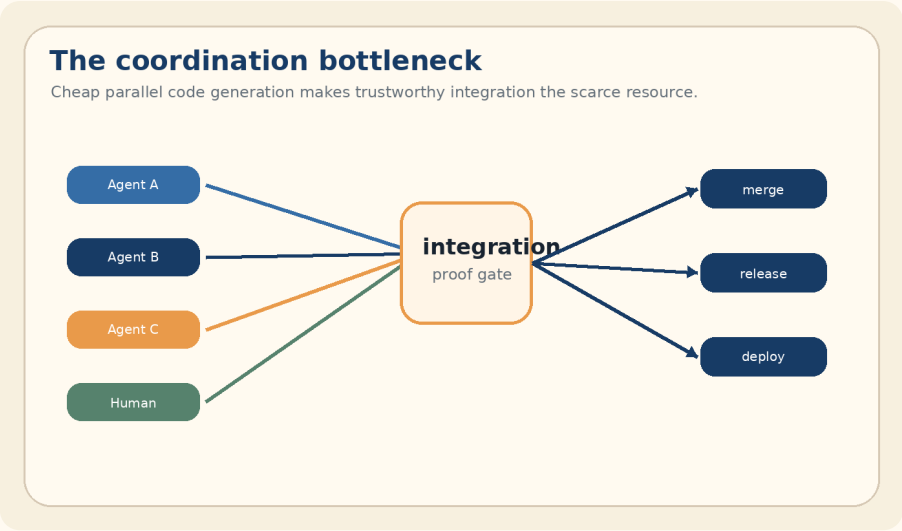

Figure 2 — The new bottleneck. Once multiple agents can produce code simultaneously, the scarce resource becomes trustworthy coordination and review rather than raw code generation.

REPOSITORY SNAPSHOT

2. RAGBAZ Frog today

The current public repository is already a serious coordination prototype rather than a toy. The package identifies itself as `ragbaz-frog 0.2.0`, requires Python 3.11+, exposes `frog` and `frog-mcp` entry points, and has an optional MCP dependency. Its master branch currently points to commit `debf6e2` from 13 June 2026, which added MCP health and gateway helpers.

Local-first

COORDINATION STATE

SQLite with explicit migrations, WAL mode, busy timeout and a guard against network-filesystem DB placement.

Agent-aware

IDENTITY & OWNERSHIP

Agent/session identity, claims, locks, stale/expired lease handling and pre-edit lock checks.

Graph-aware

TASKS & BUILDS

Task dependencies, next-task scheduling, repo dependency edges, affected builds and runner delegation.

What is already there

AREA	CURRENT FROG CAPABILITY	WHY IT MATTERS
CLI / structured output	Strict command grammar, human rendering and `--json`	Useful both for humans and agent/tool automation.
Coordination DB	SQLite, migrations, WAL, local-filesystem invariant	Simple deployment with explicit concurrency semantics.
Locks	Acquire, audit, reap, check-file, holder identity	Turns “please do not collide” into an enforceable workflow primitive.
Task lifecycle	Create, claim, finish, dependencies, blockers, priorities	Moves beyond a passive issue list toward scheduling.
Repository graph	Repo discovery, targets, dependency edges, affected builds	Lets agents reason about consequences, not just files.
Forensics	Event log, `log why`, `log blame`	Preserves causal context that chat histories often lose.
Federation	Boxes, SSH workspaces, aliases, event mirroring	Supports more than one machine without sharing SQLite over network FS.
Interfaces	TUI/board, MCP server, hooks, provider sync	Shows the core is already becoming a service used by multiple surfaces.

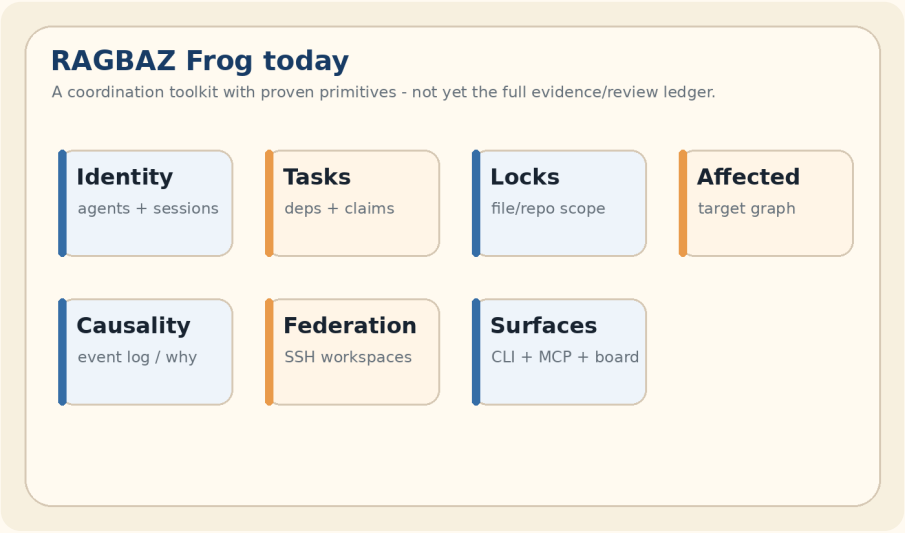

Figure 3 — Frog today. Frog already has the outline of an application service and adapter architecture. The opportunity is to make the core product concepts more explicit and reduce environment-specific assumptions.

Repository-grounded gaps before calling it an open-source product

The public repository is visible, but the current root does not expose a top-level `LICENSE` file. That is a legal and adoption gap: “public source” and “open source” are not the same thing. A license decision should therefore be an early productization task.

The development history is also still primarily direct-commit oriented. I did not find a current GitHub Actions workflow in master, and the repository search currently returns no pull requests. Frog has strong local test discipline — the commit history documents hundreds of passing unit tests — but WeftMark should dogfood the review-and-evidence model it is trying to sell.

Finally, some operational helpers still contain environment-specific assumptions, while the core package description remains tied to `/data/src`. That is entirely reasonable for an internal system, but a reusable product needs a stronger distinction between **core domain**, **local deployment profile** and **integration adapters**.

OBSERVED WORKFLOW

3. What our way of working taught us

The most useful design input does not come from imagining every possible feature. It comes from observing the repeated friction in real multi-agent work. Several patterns recur:

WHAT HAPPENS IN PRACTICE	WHAT THE PRODUCT SHOULD LEARN
A large goal is decomposed into named, dependency-aware work slices.	Make the **work graph** first-class, not merely task rows.
Different agents work on different slices and can diverge from the base branch.	Bind a claim to **branch/worktree + base SHA + scope**.
Work gets handed from one agent or vendor to another.	Create a durable **handoff object** containing state, evidence and next action.
A task can be implemented but not yet verified, reviewed or safe to merge.	Represent **evidence states** explicitly; “done” is not enough.
Agents can touch different files while modifying the same protocol or boundary.	Add **semantic / contract scopes** in addition to file locks.
Human review ends with approve, block, conditional approval or “revalidate”.	Persist the **review decision** as data, not only as a chat message.
Plans and documentation drift from what the code actually proves.	Generate human-facing status from **machine-readable facts** where possible.

> **The key shift:** Frog coordinates *work*. WeftMark should coordinate *change with proof*.

PRODUCT DIRECTION

4. WeftMark: the product thesis

WeftMark combines two useful metaphors. The *weft* is the thread woven across the warp: parallel work crossing the same codebase. A *mark* is a persistent trace, record or sign of provenance. The name therefore fits a system whose purpose is to let many workers contribute without losing the lineage of what happened.

WeftMark should be deliberately narrower than a coding agent. It does not need its own frontier model, code-edit loop, IDE or generalized cloud runtime. Those layers are already highly competitive and fast-moving. Instead, WeftMark can sit above them and make the work **legible, reviewable and portable across agent vendors**.

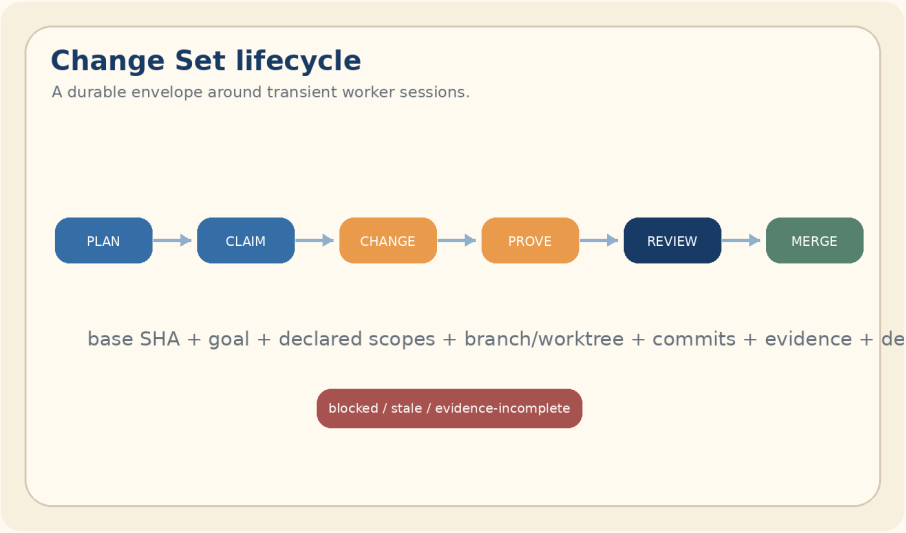

Figure 4 — The Change Set lifecycle. A change set becomes the stable envelope around transient agent sessions. Agents may come and go; the change set retains scope, lineage, evidence and decisions.

DOMAIN MODEL

5. Core concepts

5.1 Change Set

A **Change Set** is the missing object between “task” and “pull request”. It binds intent to execution state. At minimum it should know the repository, base revision, working branch/worktree, assigned agent session, declared scopes, commits, evidence requirements and current review state.

```
change_set: id: chg_01... task: ingress-mtls-proof repo:
tabenius/example base_sha: 7c3... branch: weft/ingress-mtls-proof
worktree: /.../.weft/worktrees/chg_01 agent: codex-session-42 scopes: -
file: src/ingress/** - contract: tenant-authentication evidence_policy:
- unit-tests - integration-two-host - security-review review_state:
evidence_incomplete
```

5.2 Evidence as data, not prose

Evidence needs a richer state model than pass/fail. An agent can fail a test, a CI system can be unavailable, a previously passing result can become stale when the commit changes, and a benchmark can be valid only for one machine profile.

Evidence kinds

unit test · integration test · CI · benchmark · security review · artifact · deployment probe

Evidence states

declared · running · passed · failed · unavailable · stale · superseded

5.3 Semantic scopes

File locks are necessary but insufficient. A semantic scope lets a repository declare named contracts or boundaries that cross file paths. A task that changes an authentication contract, event schema or migration protocol claims that contract even if its edited files do not overlap another task.

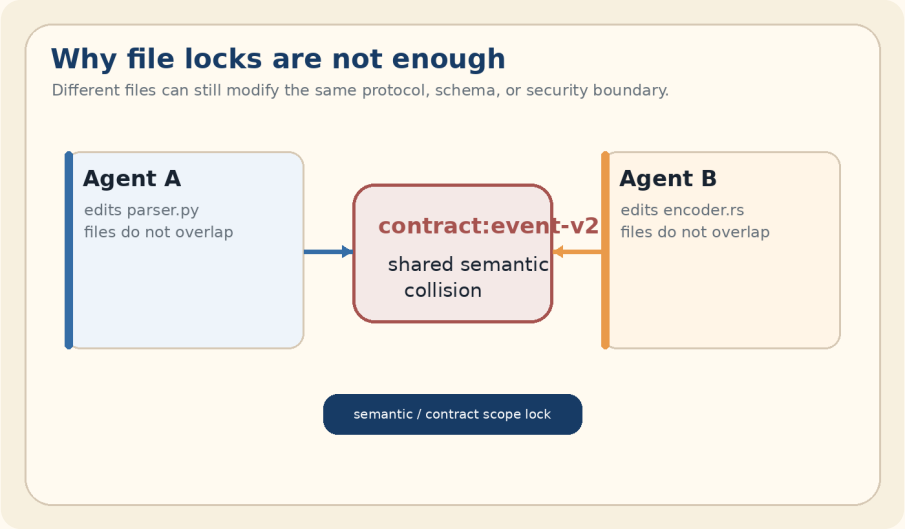

Figure 5 — Semantic collision. Three agents may touch different files while altering the same behavioral contract. Semantic locks make that hidden coupling visible.

5.4 Handoff

A handoff is a deliberately compact replay package for another agent or human. It should be generated from the ledger rather than written from memory. The receiving worker should be able to accept it and continue with the exact branch, base, active scopes, decisions, failing tests and next action.

```
weft handoff create ingress-mtls-proof --to opencode HANDOFF h_82f1
change-set chg_01a7 base/head 7c3... → 91f... active scopes
contract:tenant-auth, src/ingress/** evidence unit:passed ·
integration:failed · CI:unavailable blocked by test certificate expiry
fixture next action fix fixture; rerun integration-two-host review not
requested
```

5.5 Review as a first-class state transition

Review should answer a defined question: *does the available change and evidence satisfy the declared acceptance and safety constraints?* A useful machine-readable result is more precise than a generic “approved”.

READY | READY WITH FOLLOW-UP | BLOCKED | STALE — REVALIDATE | EVIDENCE INCOMPLETE

TARGET DESIGN

6. Proposed WeftMark architecture

The strongest architectural move is to formalize the separation that Frog has already begun: interfaces should call application services; application services should express domain rules; adapters should isolate Git, GitHub, agent runtimes, CI systems and project trackers; storage should preserve an append-friendly coordination and evidence ledger.

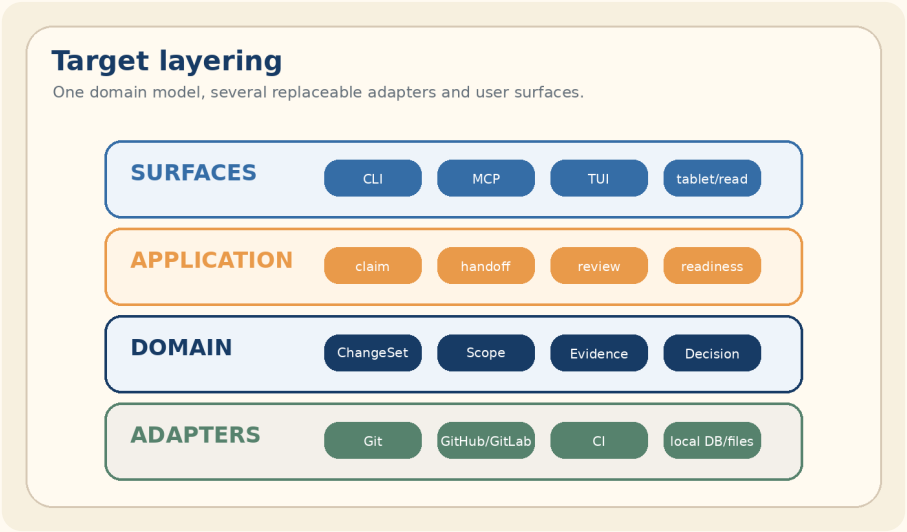

Figure 6 — Target layering. The domain layer should know that evidence exists and that a claim owns scopes; it should not know whether the worker is Cline, OpenCode, Codex, an Ollama-backed local agent or a human terminal session.

Storage direction

SQLite remains a good default for a local-first solo/small-team tool. Frog has already put useful guardrails around it. The product should resist prematurely turning the coordination database into a distributed database. A simpler path is:

1. keep a single write authority per coordination domain;
2. use append-friendly events for audit and replication;
3. store large artifacts outside SQLite and keep content-addressed references;
4. federate through explicit APIs/SSH/HTTP rather than network-mounted database files;
5. add PostgreSQL only if a real multi-user server deployment requires it.

Protocols: use rather than invent

Two open protocols are especially important. **MCP** standardizes how agent applications reach tools and context. **ACP** standardizes how editors/clients communicate with coding agents and explicitly targets both local and remote scenarios. WeftMark should therefore avoid creating a proprietary “universal agent protocol”. It can implement adapters around MCP/ACP and focus its own API on coordination, provenance and evidence.

ENGINEERING

7. Implementation perspective

The migration should be evolutionary. Frog has working behavior and a substantial test suite; replacing it wholesale would destroy the most valuable asset — the encoded lessons of actual use.

7.1 Start by dogfooding the process

Before implementing advanced features, WeftMark/Frog should adopt the workflow it wants to enforce:

```
task → claim → branch/worktree → implementation → evidence → PR →
review → merge
```

The first infrastructure work should therefore add a public CI workflow, an explicit open-source license, reproducible package/test commands, basic release workflow, and a protected “PR before merge” habit for WeftMark itself. This gives the project a real evidence surface to integrate.

7.2 Introduce new tables without breaking old commands

NEW ENTITY	MINIMAL FIELDS	MIGRATION STRATEGY
change_sets	id, task_slug, repo_id, base_sha, head_sha, branch, worktree, state	Create lazily when a claimed task gains a branch/worktree.
scopes	id, kind=file|contract|schema|boundary, key, repo_id	Map existing lock file patterns to file scopes.
claims	change_set, agent, session, lease, active	Reuse current atomic claim semantics.
evidence	kind, subject, commit, status, producer, command, environment, artifact_ref	Initially ingest local test and CI results.
reviews	change_set, reviewer, verdict, findings, evidence_snapshot	Manual first; provider adapters later.
handoffs	change_set, from/to, summary, next_action, created_at	Generated projection from current state.

7.3 Keep a compatibility bridge

A safe product rename does not require immediately breaking the existing CLI. A practical sequence is to ship the package as WeftMark while keeping `frog` as a compatibility entry point for one major release. Commands can gradually move from implicit “task finished” semantics to explicit change-set/evidence semantics.

```
frog task claim foo # legacy-compatible weft task claim foo # new
name weft change show foo weft evidence run --required weft review foo
weft handoff create foo --to cline
```

7.4 Split modules by domain, not by arbitrary size

Earlier Frog work deliberately avoided a risky physical split of the large storage module and instead introduced lightweight domain facades. That was a sensible stabilization decision. The new domain objects now provide a better seam for the next refactor:

```
weftmark/ domain/ tasks.py scopes.py evidence.py changesets.py
reviews.py handoffs.py application/ scheduler.py change_service.py
review_service.py adapters/ sqlite.py git.py github.py agent_acp.py
mcp.py ci.py interfaces/ cli/ tui/ http/
```

The crucial rule is not the folder names. It is that **CLI, MCP, TUI and a future web/tablet client all invoke the same application services**.

7.5 Security and trust boundaries

As agents become more autonomous, WeftMark will hold sensitive power: it can assign work, open worktrees, start processes, collect credentials indirectly through adapters and decide whether evidence is sufficient. The design should therefore default to least privilege:

- agent runners get only the repository and tools needed for the claimed scope;
- credentials remain in provider-specific stores; WeftMark stores references, not secrets;
- remote execution is authenticated and auditable;
- evidence producers are identified so self-attestation can be distinguished from independent verification;
- release/merge operations remain separate capabilities from code editing.

OPEN-SOURCE LANDSCAPE

8. Where the open-source tooling ecosystem is now

The ecosystem is converging on a useful division of labor. Open-source coding agents are becoming provider-agnostic and accessible through multiple clients. Separate orchestration tools are appearing to run many agents in parallel. Protocols are reducing editor/tool lock-in. This is exactly the environment in which a neutral coordination-and-evidence layer can make sense.

PROJECT	OPEN-SOURCE ROLE	RELEVANT SIGNAL FOR WEFTMARK
OpenCode	Provider-agnostic coding agent with TUI and client/server architecture.	Agent frontends can be decoupled from execution; remote/mobile control becomes natural.
OpenHands	Agent SDK plus local/remote execution model, Docker runtime and server APIs.	Sandboxed and remote execution are becoming reusable infrastructure, not features every coordinator must reinvent.
Cline	Open-source SDK, CLI and IDE agent with MCP, checkpoints, subagents/teams and provider choice.	Agent harnesses themselves are becoming embeddable platforms.
Cline Kanban	Parallel agent task board; each task gets an ephemeral Git worktree, diff review and dependency chaining.	Worktree-per-task and human review are emerging as common orchestration patterns.
Aider	Terminal pair programmer with strong Git integration and broad model support.	A coordination layer should treat Git-native agent workflows as first-class, not force a proprietary execution model.
Goose	Local desktop/CLI/API agent, multiple providers, MCP extensions and ACP integration.	Open agent clients are becoming multi-provider and multi-surface by default.
Continue	Open-source CLI increasingly focused on source-controlled AI checks enforced in CI.	Verification agents and code-generation agents are separating — evidence becomes its own workflow.
Claude Squad	TUI for multiple coding agents in isolated Git workspaces.	Solo developers increasingly need session orchestration across vendors.
Agent Deck	Terminal session manager supporting multiple agent CLIs and Git worktrees.	The “mission control” category is real, but session management alone does not preserve durable proof.
ACP + MCP	Open protocols for agent↔client and model/app↔tool interaction.	WeftMark can integrate instead of becoming another closed agent ecosystem.

Important market lesson. Roo Code was a major open-source agent project and was archived in May 2026. Vibe Kanban, another prominent multi-agent orchestration project, has announced a sunset. Fast project churn is not a reason to avoid the category; it is a reason to keep WeftMark's durable state independent of any one agent frontend.

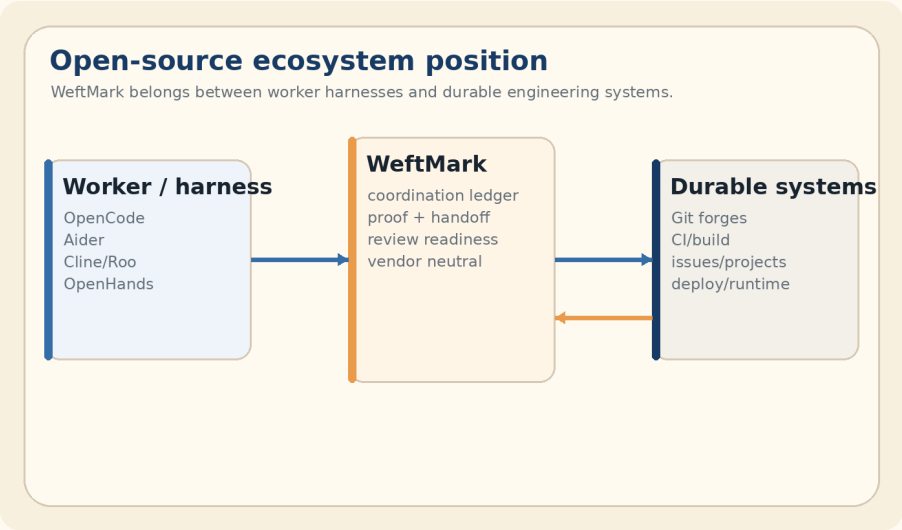

Figure 7 — Ecosystem position. The open-source category is already rich in agent loops and parallel UIs. WeftMark's differentiation is durable state and evidence across them.

DISCUSSION

9. Where solo and small-team multi-agent work is going

This section is intentionally interpretive. The following points are inferences from current open-source project direction, protocol work and the workflow patterns described earlier.

9.1 The unit of work is moving from “chat” to “workspace”

A chat transcript is not enough to manage parallel engineering. Current orchestration tools increasingly give each task a branch or worktree, its own terminal, and a reviewable diff. The durable unit is becoming a **workspace with lineage**. WeftMark should make that lineage explicit and agent-neutral.

9.2 The human becomes scheduler and reviewer

As agents become faster, a solo developer can behave more like a small engineering manager: decompose work, start several independent threads, inspect failures, compare approaches, and decide what enters the main branch. This makes review ergonomics, prioritization and “what needs my attention now?” more valuable than another chat window.

9.3 Multi-vendor becomes normal

Provider-agnostic tools such as OpenCode, Cline, Aider and Goose already assume that the “best model” is not permanent. Some users will route a planning task to one model, a mechanical refactor to another, and a private repository analysis to a local model through Ollama. WeftMark should identify the worker and record provenance without making model choice part of its core policy.

9.4 Protocols reduce client lock-in, not coordination complexity

MCP and ACP make it easier to connect tools, context, editors and agents. That is good, but standardized connectivity does not answer ownership, dependency, merge readiness or evidence freshness. Protocol convergence therefore increases the number of interchangeable workers — and makes a neutral coordination layer more useful, not less.

9.5 Worktree isolation becomes a baseline; semantic isolation is next

Git worktrees are appearing repeatedly because they are cheap, understandable and native to the underlying version-control system. They solve many filesystem collisions. The next frontier is semantic scope: contracts, schemas and boundaries that are shared even when files are separate.

9.6 Evidence will become a competitive boundary

When code production is cheap, confidence becomes expensive. A small team will increasingly care about whether an agent can provide reproducible test evidence, whether a reviewer is independent from the implementer, whether CI actually ran, and whether the result is stale after a rebase. Continue's move toward source-controlled AI checks is one visible signal in this direction.

9.7 Open-source control planes have a structural advantage

The coordination ledger contains intimate knowledge of a developer's repositories, task priorities, failures, agent behavior and operational environment. Open source matters here for reasons beyond price: inspectability, long-lived exportability, self-hosting, offline/local use, and the ability to survive the shutdown or strategic pivot of an agent vendor.

Design implication. WeftMark should optimize for *replaceable workers and durable work*. The agent session is temporary; the change set, evidence and review record are the asset.

ROADMAP

10. A practical Phase III roadmap

Phase III should be deliberately narrow. The project already has many features; the risk is feature accumulation without a sharper product boundary.

0

Open-source and evidence baseline

Add a recognized open-source license, GitHub Actions CI, reproducible package/test commands, basic release workflow, and a protected “PR before merge” habit for WeftMark itself.

Exit: every master change has a reviewable change record and machine-generated evidence.

1

Change Set model

Add branch/worktree/base-SHA lineage around existing task claims. Keep old task commands working.

Exit: `weft change show` can explain exactly what workspace and commits implement a task.

2

Evidence ledger

Record local test runs, CI checks and artifact references with producer, commit and freshness semantics.

Exit: finishing a task can require evidence policy rather than a boolean status update.

3

Handoff + review

Generate compact handoff packages and persist reviewer verdicts/findings. Provide human-first rendering in CLI/TUI before building a full web UI.

Exit: a new agent can accept work without reconstructing context from chat logs.

4

Semantic scopes

Add repository-defined contract/schema/boundary scopes and conflict detection. Use them in scheduler and pre-edit hooks.

Exit: two tasks touching the same architectural contract cannot silently run as independent work.

5

Agent adapters and open protocols

Stabilize a runner adapter around CLI/ACP-capable agents. Treat Cline, OpenCode, Goose, Aider and others as replaceable workers. Continue using MCP for tools where appropriate.

Exit: the same WeftMark task can be handed between two different agent vendors without losing lineage.

6

Tablet/web review surface

Only after the data model is stable, build a read/review-focused web client: queues, diffs, evidence, handoff and review decisions. Remote control should be a projection of the core state, not a second workflow engine.

Exit: a developer can steer and review parallel work comfortably from a tablet without sacrificing the CLI.

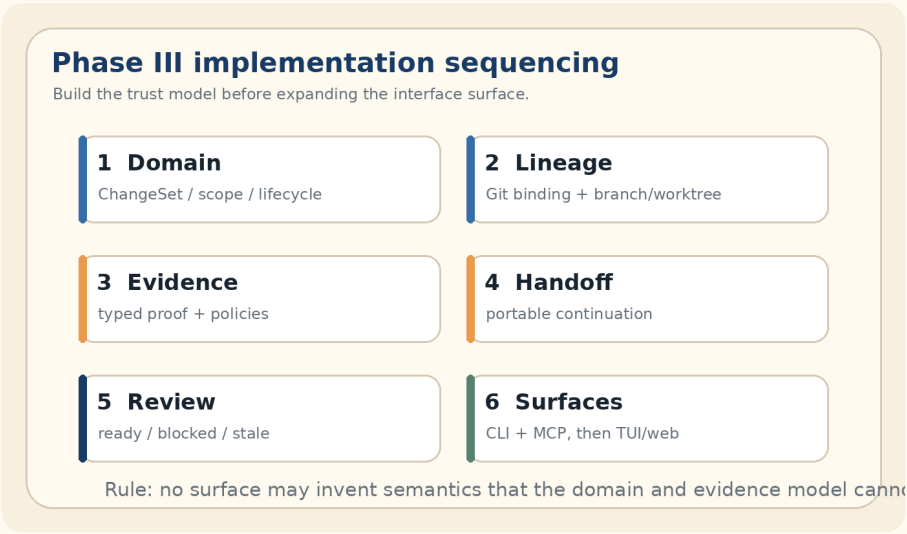

Figure 8 — Phase III sequencing. The highest leverage work is data model and evidence discipline. A web/tablet surface becomes much easier once the same state already serves CLI, TUI and MCP.

CONCLUSION

11. A sharper product, not simply a larger Frog

Frog has already demonstrated the difficult part: a small coordination system can encode real operational lessons about concurrency, identity, task claiming, remote workspaces, dependency-aware scheduling and forensic history. The next step is to give those lessons a stronger product boundary.

WeftMark should be the layer that survives agent churn. A developer may use Claude Code for one task, Codex for another, OpenCode with a Zen model for a third, and a local Ollama-backed agent for sensitive work. The exact workers will change. The **task graph, declared scope, branch lineage, evidence, review and handoff** should not.

> **North-star statement.** WeftMark is the ledger that can explain exactly why an agent was allowed to change something, what it changed, what evidence exists that it works, who reviewed it, and whether it is actually safe to merge.

That is a more defensible niche than becoming another coding agent, another Kanban board or another wrapper around a model API. It is also closely aligned with where open-source agent tooling appears to be heading: interchangeable workers, open protocols, isolated workspaces, remote execution, and increasingly formal review and verification.

SOURCES

References

Repository observations and ecosystem descriptions were checked against the sources below on 11 August 2026. Fast-moving project status should be rechecked before publication or implementation decisions.

1. **RAGBAZ Frog repository.** Current README, PLAN, source tree, package metadata and commit history for `tabenius/frog`. https://github.com/tabenius/frog
2. **Frog PLAN.md.** Evolution plan describing the project's deliberate niche as a multi-agent coordination, scheduling and forensics layer. https://github.com/tabenius/frog/blob/master/PLAN.md
3. **Frog pyproject.toml.** Package name/version, Python requirement and CLI/MCP entry points. https://github.com/tabenius/frog/blob/master/pyproject.toml
4. **OpenCode.** Open-source coding agent; provider-agnostic positioning, TUI focus and client/server architecture. https://github.com/anomalyco/opencode
5. **OpenCode Zen.** Current Zen model catalogue, including Big Pickle. https://opencode.ai/docs/zen
6. **OpenHands Software Agent SDK.** Modular SDK for local/remote code agents and multi-agent tasks. https://docs.openhands.dev/sdk
7. **OpenHands Runtime Architecture.** Docker-based sandboxed runtime for agent actions. https://docs.openhands.dev/openhands/usage/architecture/runtime
8. **Cline.** Open-source SDK, CLI and IDE coding agent; current repository overview. https://github.com/cline/cline
9. **Cline CLI.** Current CLI features including MCP, checkpoints, sub-agents/teams, ACP mode and headless operation. https://github.com/cline/cline/blob/main/apps/cli/README.md
10. **Cline Checkpoints.** Shadow-Git checkpoint model for reversible agent work. https://docs.cline.bot/core-workflows/checkpoints

11. **Cline Kanban.** Parallel task board using per-task worktrees, diffs, dependency chains and PR flows. https://github.com/cline/kanban
12. **Aider.** Terminal pair-programming agent with Git integration and broad cloud/local model connectivity. https://github.com/Aider-AI/aider
13. **Goose.** Open-source local desktop/CLI/API agent with multi-provider support, MCP extensions and ACP integration. https://github.com/aaif-goose/goose
14. **Continue.** Open-source CLI and source-controlled AI checks enforced through CI. https://github.com/continuedev/continue
15. **Claude Squad.** Open-source terminal manager for multiple coding agents in isolated Git workspaces. https://github.com/smtg-ai/claude-squad
16. **Agent Deck.** Open-source multi-agent terminal session manager with worktree support. https://github.com/asheshgoplani/agent-deck
17. **Agent Client Protocol (ACP).** Open protocol standardizing communication between code editors/clients and coding agents, including local and remote scenarios. https://agentclientprotocol.com/get-started/introduction
18. **ACP reference implementation/specification repository.** Apache-2.0 protocol project and registry ecosystem. https://github.com/agentclientprotocol/agent-client-protocol
19. **Model Context Protocol (MCP).** Open specification and documentation for tool/context interoperability. https://github.com/modelcontextprotocol/modelcontextprotocol
20. **Roo Code.** Archived open-source coding-agent repository; extension shutdown announced 15 May 2026. https://github.com/RooCodeInc/Roo-Code
21. **Vibe Kanban.** Open-source multi-agent workspace/plan/review tool whose repository now announces that the product is sunsetting. https://github.com/BloopAI/vibe-kanban

This document is a product and engineering analysis, not a commitment to a particular license, hosting model or protocol implementation. Any open-source license selection should be made explicitly before WeftMark is presented as an open-source release.

RAGBAZ Design · From RAGBAZ Frog to WeftMark · 11 August 2026 · self-contained HTML edition